

Experiment 11-Policy Gradient

Policy Gradient Implementation using REINFORCE Algorithm

NAME: AKHILA POCHIMI REDDY

REG.NO:192425225

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SOLT:B

Aim:

To implement the **Policy Gradient method using the REINFORCE algorithm** and evaluate the agent's learning performance using rewards and visualization.

Procedure:

1. Import NumPy, TensorFlow/Keras, and Matplotlib.
2. Create a simple environment with states and two actions.
3. Build a policy network using Keras.
4. Select actions according to the probability generated by the policy network.
5. Store rewards obtained during each episode.
6. Calculate the discounted return for each step.
7. Update the policy using the REINFORCE algorithm.
8. Record episode rewards and display the training performance.
9. Plot the reward graph.

Python Code:

```
import numpy as np, tensorflow as tf, matplotlib.pyplot as plt
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
np.random.seed(1); tf.random.set_seed(1)
model=Sequential([Dense(16,activation='relu',input_shape=(4,)),
                  Dense(2,activation='softmax')])
```

```

opt=tf.keras.optimizers.Adam(0.01)
rewards=[]
for ep in range(50):
    states=[]; actions=[]; rs=[]
    s=np.random.rand(4)
    for _ in range(10):
        p=model(s.reshape(1,-1),training=False).numpy()[0]
        a=np.random.choice(2,p=p)
        r=1 if a==1 else -1
        states.append(s); actions.append(a); rs.append(r)
        s=np.random.rand(4)
    G=[]; g=0
    for r in rs[::-1]:
        g=r+0.95*g; G.append(g)
    G=G[::-1]
    with tf.GradientTape() as tape:
        loss=0
        for s,a,g in zip(states,actions,G):
            p=model(s.reshape(1,-1),training=True)[0]
            loss-=tf.math.log(p[a]+1e-8)*g
        grads=tape.gradient(loss,model.trainable_variables)
        opt.apply_gradients(zip(grads,model.trainable_variables))
    rewards.append(sum(rs))
print("==== REINFORCE SUMMARY ====")
print("Episodes:",len(rewards))
print("Average Reward:",round(np.mean(rewards),2))
print("Best Reward:",max(rewards))
print("Final Reward:",rewards[-1])
plt.plot(rewards)
plt.title("REINFORCE Training Reward")

```

```
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.grid()
plt.show()
```

Result:

The **REINFORCE Policy Gradient algorithm** was successfully implemented using TensorFlow and Keras. The summary values and reward graph were obtained to evaluate the learning performance of the policy.

Summary Output:

```
===== REINFORCE SUMMARY =====
Episodes: 50
Average Reward: 8.64
Best Reward: 10
Final Reward: 10
```

Visualization Output:

